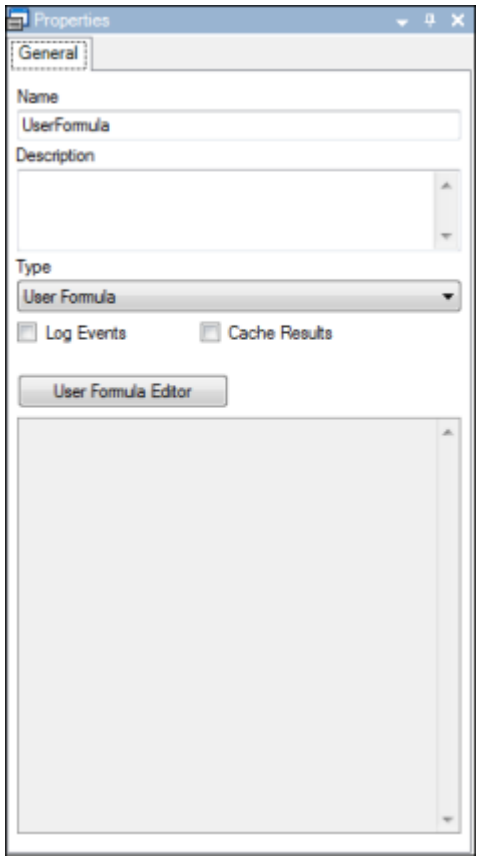
User Formula Function

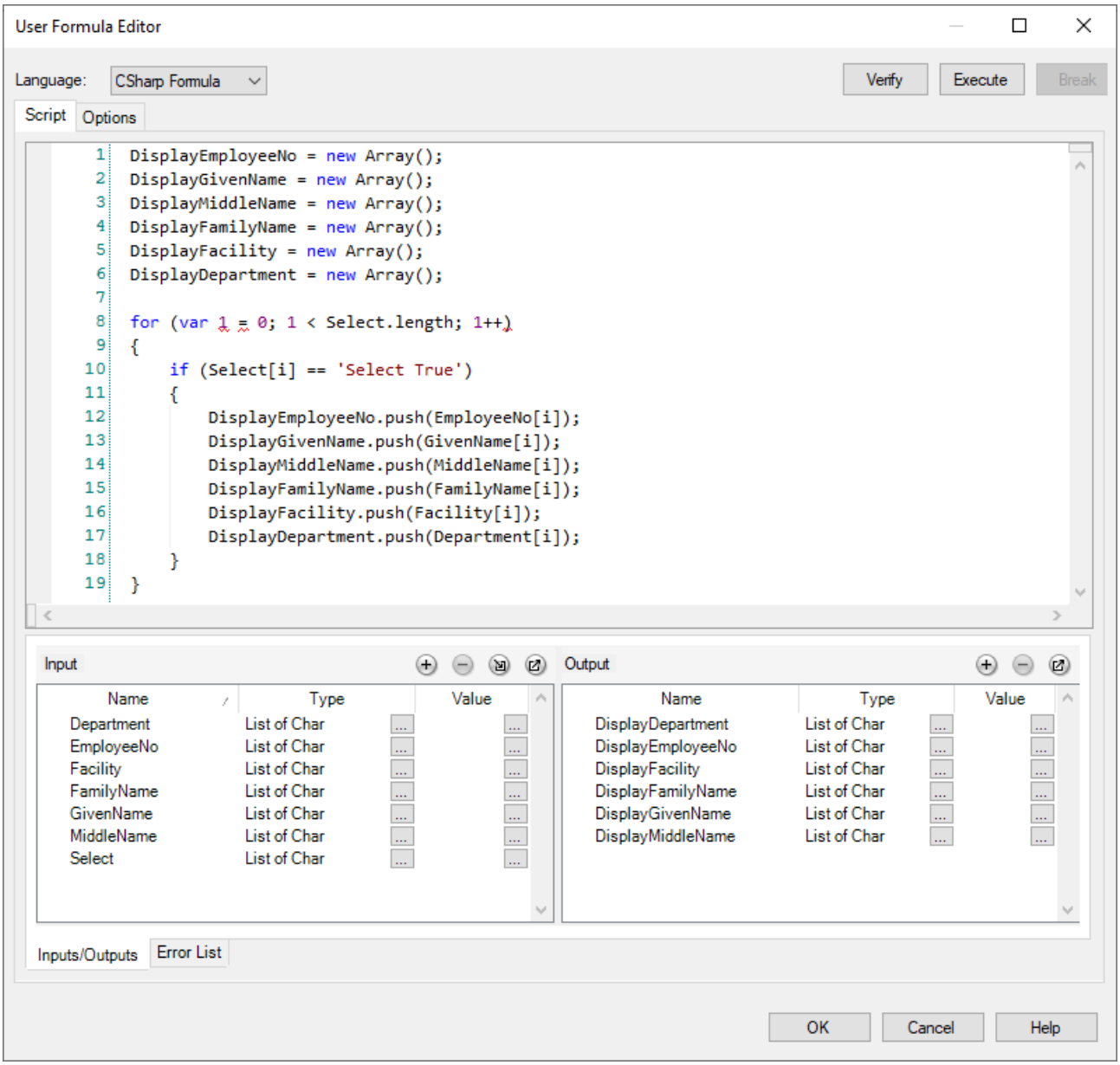
This type of Function is used to perform computations using the JScript, Visual Basic, or CSharp Formulas.

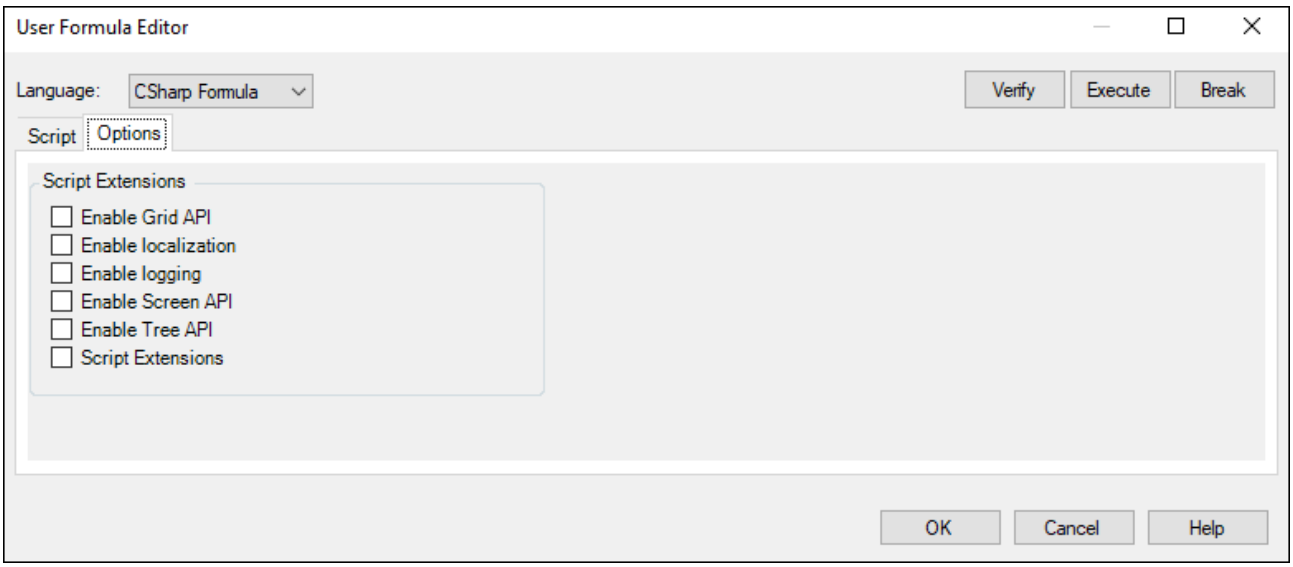
To Add the User Formula Function

1. Drag the Function User Formula from the Toolbox to the Function Navigation diagram, or double-click the User Formula icon in the Toolbox.



2. Use the **User Formula Editor** to create the Formula Code.





Field	Description
Language	The editor enables the creation of code in Visual Basic Script, JScript, and CSharp language.
Verify	Validates the script.
Execute	Executes the script.
Break	Breaks the execution of the script.

Script

Allows the user to edit the formula code. The syntax of the formula is highlighted and the code lines are numbered. The user has access to all the actions in the text like copy, paste, undo, etc. It is also possible to find expressions in the formula code using the `CTRL+F` shortcut (see also: Keyboard Shortcuts).

> The CSharp code created within the User Formula Function should only perform in-memory calculations. Actions that require file, database, network, or other input/output permissions are not recommended (use them with caution). Code containing such actions will pass validation, but errors will be displayed at runtime.

> `System.Collections.Generics`, `System.Linq`, `System.Text.RegularExpressions`, and `System.XML` namespace may be used in the User Formula Editor (CSharp and Visual Basic languages).

Options

The Options tab contains a Script Extensions section, which includes the options below:

Enable Grid API

Adds a reference to Grid Control assemblies in the User Formula code (see Export to User Formula option in the Grid Control).

Enable localization

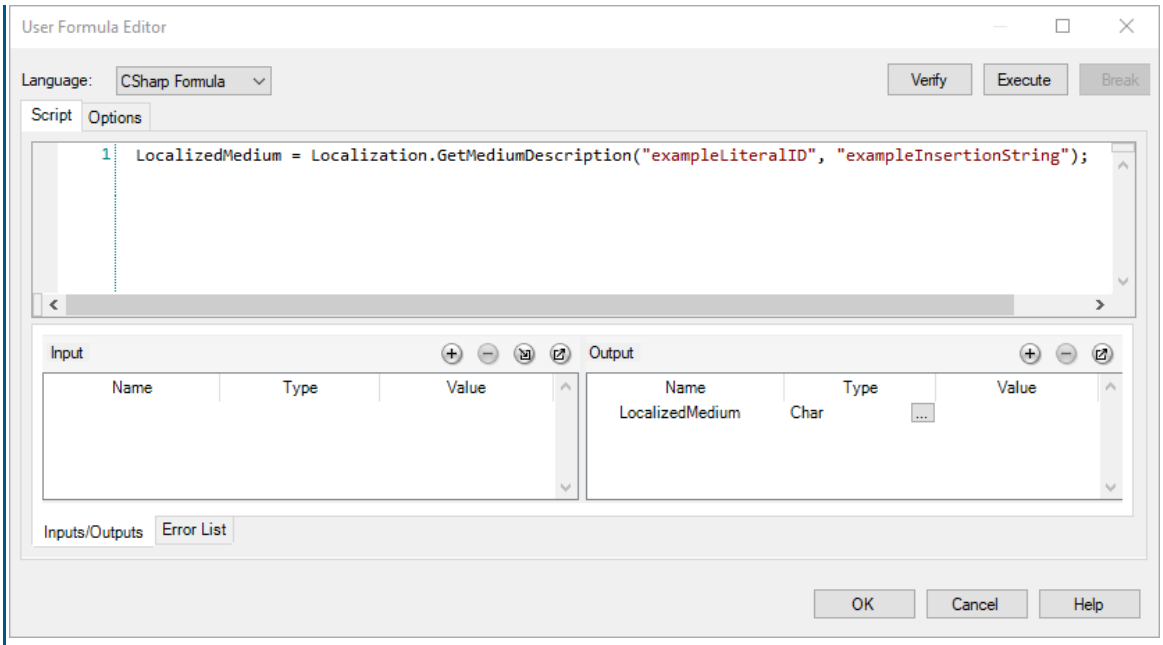
Enables the use of localized literals. You can use the Localization object in the CSharp script. The following methods are available:

```
public string GetShortDescription(string literalID, params string[]
insertionStrings)
```

```
public string GetMediumDescription(string literalID, params string[]
insertionStrings)
```

```
public string GetLongDescription(string literalID, params string[]
insertionStrings)
```

The example below returns the medium description of the literal with the ID of "exampleLiteralID" (from the LITERAL table). The language depends on the user session settings (stored in the @LanguageID variable).



The "PermissionSet" key must be set to FullTrust (for details, refer to the "ScriptManagerSettings" section of the Central Configuration Documentation).

Enable logging

When selected, this enables sending additional information to the Function Interpreter log files. This option is only available for Visual Basic and CSharp formulas. For details on the logging framework, see ***Logging Guide***. To trigger the logging action from a script, the "Log" object must be used. Separate methods are available for logging events of different severity:

- Debug(), Error(), Fatal(), Info(), Warn()

```
Log.Debug("Product not found.");
```

- DebugFormat(), ErrorFormat(), FatalFormat(), InfoFormat(), WarnFormat()

```
Log.WarnFormat("Container {0} has invalid status {1}.",
ContainerNo, ContainerStatus);
```

The priority of the log message is determined by the method used. Based on priority, the log message will be appended to the appropriate log file according to the logging configuration. The method will log the message and an optional exception that was caught earlier.

Enable Screen API

The Screen API enables access to Screen Flow-related methods that allow you to access the Screen Flow entity configuration directly (for Screens, Views, and Actions) and to write custom rendering logic.

Screen API is used only in custom/extension scenarios.

After enabling the Screen API, all the available classes (for example, `SfHelper`) are added to the IntelliSense code editor. For a grid or tree, there is no

	IntelliSense, so the user should use some of these methods.
	
	Enable Tree API
	
	Adds a reference to the Tree Control assemblies in the User Formula code (see Export to User Formula option in the Tree Control).
	
	Script Extensions
	
	Enables the assemblies defined in `FlexNet.ScriptExtensions` to be used in C# script:
	
	`<FlexNet.ScriptExtensions>`
	`<!-- UserFormula Function extensions, registered assemblies/namespaces can be used in a C# formula.-->`
	`<!-- add key="ExtensionName" value="DefaultNamespace, AssemblyName" /-->`
	`</FlexNet.ScriptExtensions>`

Inputs/Outputs tab	A list of the script's Inputs and Outputs. You can add an Input or Output using one of the methods below: ▶ Using ⊕. ▶ Using the right-click menu on the selected text in the code editor. ▶ Using the `Ctrl+Shift+I` (for Inputs) and `Ctrl+Shift+O` (for Outputs) Keyboard Shortcut. Remove Inputs and Outputs from the list by using ⊖. Change the data type by clicking **...**. For the List data type on the Inputs tab, you can add and remove values in the **Value Editor** by clicking **...**. On the Outputs tab, the **Value Editor** is read-only. The **Value** field (with values used for testing purposes only) might encounter issues with recognizing certain standard encoding characters, for example ASCII code 29 (GS).
Error List tab	Contains errors triggered during verification and execution of the script. Error descriptions can be copied using the right-click menu.

If your User Formula script is resource-intensive (for example, involves processing very large amounts of data), it might fail to run properly.

Using Custom Extensions inside User Formula (CSharp User Formula only)

It is also possible to invoke custom C# code inside User Formula.

1. Copy a DLL file to GAC and `<drive>\Program Files\Dassault Systemes\DELMIA Apriso 2025\WebSite\Downloads\PB2` (on the server).
2. Register the DLL file in the "ScriptExtensions" section of the Central Configuration file. Specify an "ExtensionName" and its corresponding values (for details, see Central Configuration Documentation) as in the example below:

```
<add key="Test" value="FlexNet.Test, FlexNet.Test, Version=9.6.0.0, Culture=neutral,
PublicKeyToken=33f692765842122b" />
```

3. Rebuild **ClickOnce** and restart **Apriso Services/IIS**.

Using Dates in the User Formula Function

The formula's DateTime variables are always stored with local time. Provide the Input values in your local time zone, and the formula will return values in the local time zone, too. Some usage examples of the DateTime variables in the JScript formula language are:

- `InDateTime` – Input value
- `OutDateTime` – Output value
- `OutDateTimeString` – Output string value

1. The new variable of the DateTime type will contain the current local time:

```
OutDateTime = new Date();
```

> It is recommended to use the appropriate System Variables to retrieve the current local or UTC time.

2. Create a custom string representing time in the local time zone:

```
var Month   = InDateTime.getMonth();
var Day     = InDateTime.getDate();
var Year    = InDateTime.getFullYear();
var Hours   = InDateTime.getHours();
var Minutes = InDateTime.getMinutes();
var Seconds = InDateTime.getSeconds();
OutDateTimeString = Year + '/' + Month + '/' + Day + ' ' +Hours + ':' + Minutes +
':' + Seconds;
```

3. Create a custom string containing time in the UTC representation:

```
var Month   = InDateTime.getUTCMonth();
var Day     = InDateTime.getUTCDate();
var Year    = InDateTime.getUTCFullYear();
var Hours   = InDateTime.getUTCHours();
var Minutes = InDateTime.getUTCMinutes();
var Seconds = InDateTime.getUTCSeconds();
OutDateTimeString = Year + '/' + Month + '/' + Day + ' ' +Hours + ':' + Minutes +
':' + Seconds;
```

Using Messages in the User Formula Function (CSharp User Formula only)

You can display the error message while using the User Formula Function (`OperationException` object). The message can be provided:

- directly

```
throw new OperationException("My Exception {Input1}", "");
```

- by entering the LiteralID from the LITERAL table (Literals need to be manually added to the database)

```
throw new OperationException("", "LiteralID");
```

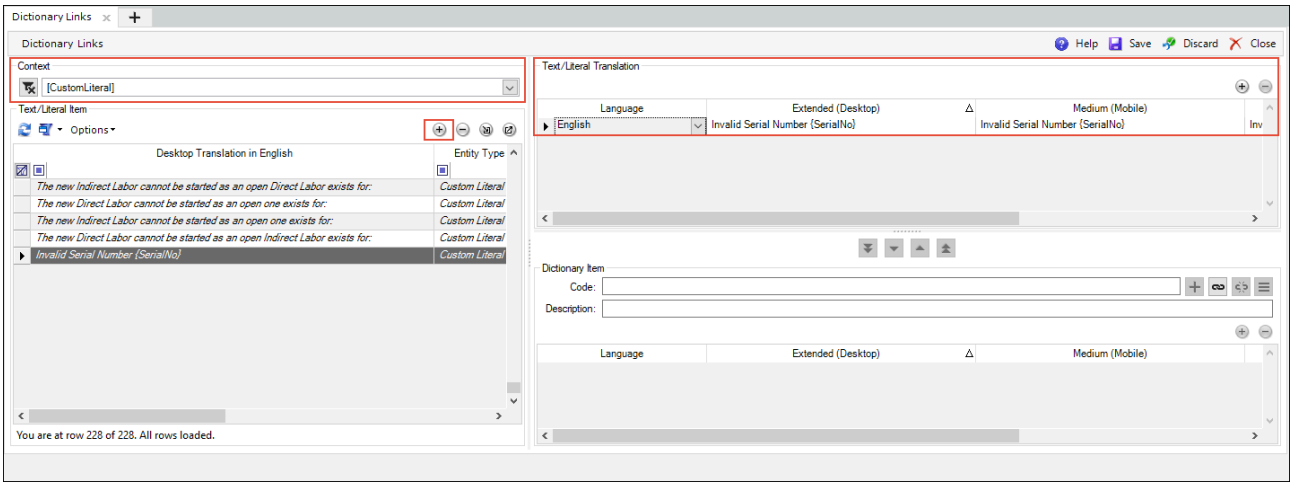
You can also use the "**showModal**" property in the `OperationException` object to determine whether the message should be shown in a modal pop-up window.

```
throw new OperationException(string message, string LiteralID, bool showModal)
```

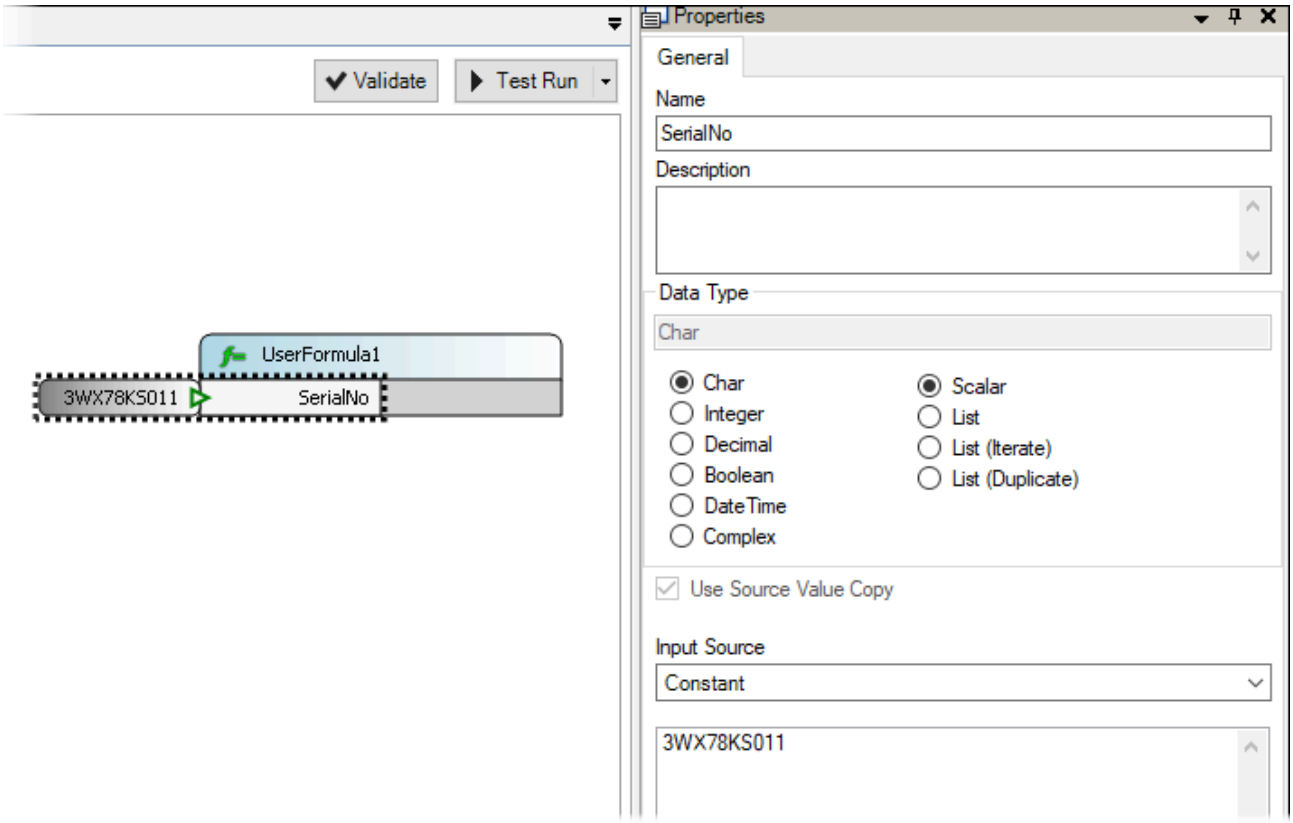
Parameters (inside `{ }` brackets) are replaced with the User Formula Function Inputs or parameters provided in the Exception constructor. Inputs can be overridden using custom parameters with the same name.

Example

1. Go to Dictionary Links (see Dictionary screen) and choose the "Custom Literal" filter.
2. Click ⊕ in the Text/Literal Item grid.
3. Provide a Literal Code (for example, "ErrorMessageInvalidSerialNumber") and click "Add".
4. Enter "Invalid Serial Number {SerialNo}" as the Translation in the Text/Literal Translation grid.

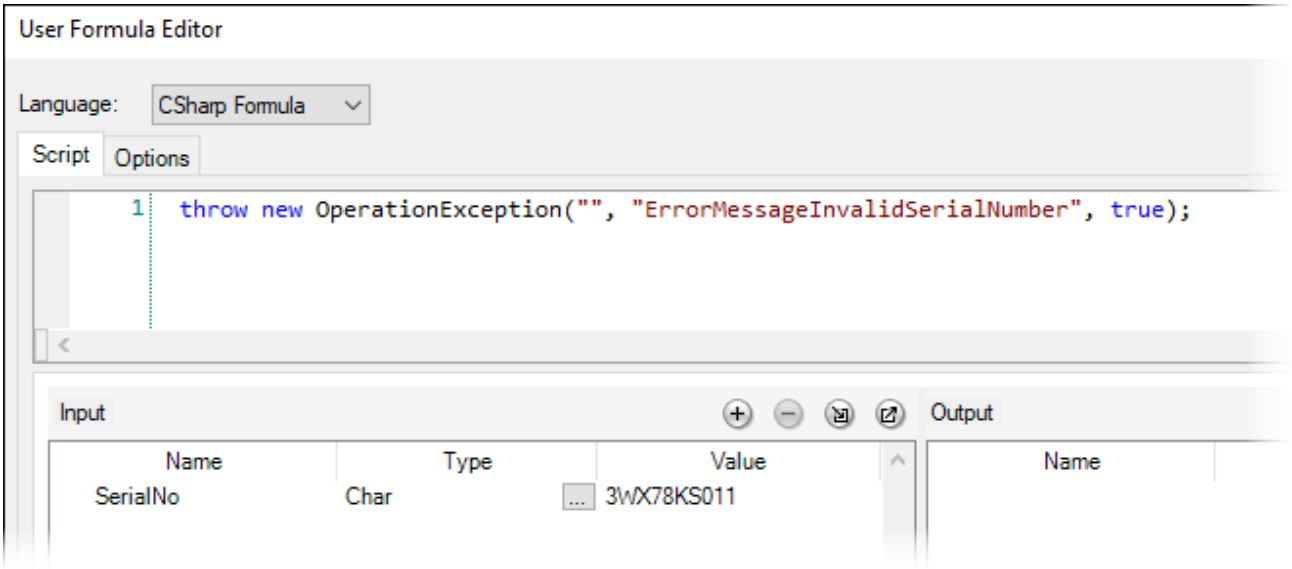


5. Create a DFC and add one Step and a User Formula Function.
6. Add an Input and set its name to "SerialNo" (Input Source: Constant, Value: **3WX78KS011**)

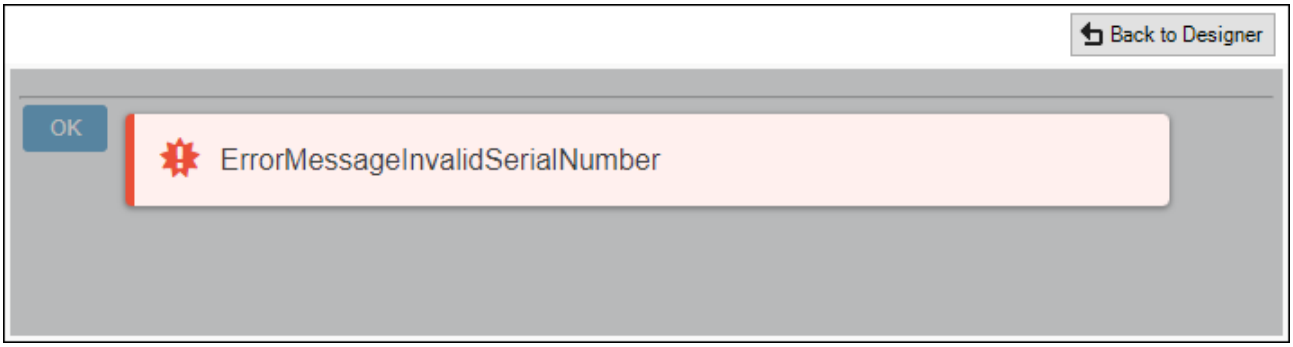


7. Open the User Formula Editor, select **CSharp Formula** in the Language drop-down menu, and enter a script (use the "**showModal**" property).

```
throw new OperationException("", "ErrorMessageInvalidSerialNumber", true);
```



8. Start Test Run.



Advanced Usage

An additional constructor is available for the `OperationException` class, which allows passing custom parameters created inside the User Formula:

```
OperationException(string message, string LiteralID, Dictionary<string, object> parameters)
```

Advanced Usage Example

LiteralID: `MyErrorMessage`; value: "`My Message With Parameters {param1}, {param2}`"

Possible implementations:

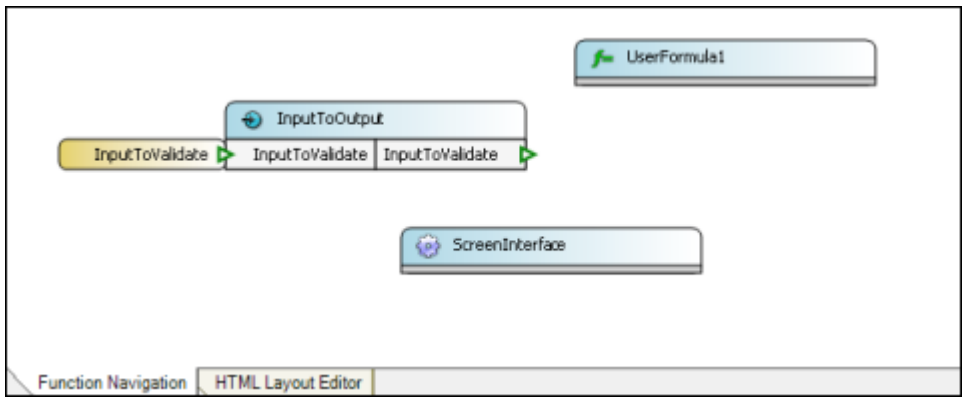
```
throw new OperationException("", "MyErrorMessage", new Dictionary<string, object>{{"param1",
"value1"}, {"param2", "value2"}});
```

or:

```
var dict = new Dictionary<string, object>();
dict.Add("param1", "value1");
dict.Add("param2", "value2");
throw new OperationException("", "MyErrorMessage", dict);
```

Displaying a Standard Error Message in a Custom Way

1. Create a DFC with Input to Output and User Formula Functions.
2. Convert to HTML Layout (use HTML Layout Editor Right-Click Menu option).



3. Add the following code to the User Formula Editor (CSharp Formula Language).

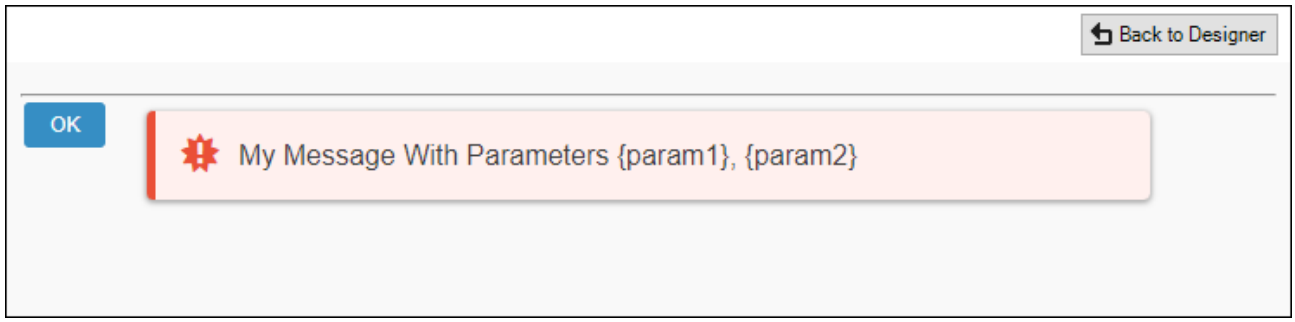
```
throw new OperationException("My Message With Parameters {param1}, {param2}", "");
```

4. Put the JScript code (presented below) into the JavaScript tab (HTML Layout Editor).

```
window.showError = function(error, originalFunction) {
//Shows Error Message alternatively in JavaScript alert window
alert(error.Content);
//Use line below to execute Error Message in a standard way

//originalFunction();
}
```

5. Start Test Run.


Back to Designer
OK
My Message With Parameters {param1}, {param2}